Zephyr Project

IT-Schulungen.com
Technology & Transformation

Eigene Shell-Kommandos sicher entwerfen

Kommandovertrag, Validierung und Delegation · native_sim

3.06

Bausteine eines Shell-Kommandos

01	`Root-Kommando`	event bildet den stabilen Einstiegspunkt der Anwendung.
02	`Unterkommando`	status, stats und reset trennen fachliche Bedienfälle.
03	`Handler`	Der Handler prüft Eingaben, delegiert und liefert eine klare Antwort.

Hilfetext, Argumentvertrag und Rückgabewert gehören zur öffentlichen Shell-Schnittstelle.

Kommandovertrag vor dem Code

SYNTAX

Aufruf eindeutig

Name, Pflichtargumente und optionale Argumente vorab festlegen.

WERTEBEREICH

Eingaben begrenzen

Erlaubte Zahlen, Einheiten und Grenzwerte dokumentieren.

SEITENEFFEKT

Wirkung festlegen

Status lesen, Zustand ändern oder asynchrone Arbeit starten.

Der Vertrag beschreibt Erfolg, Fehler und Zustandsänderung aus Sicht der Bedienperson.

Registrierung und Unterkommandos

SHELL-REGISTRIERUNG

```
SHELL_STATIC_SUBCMD_SET_CREATE(sub_event, ...);
SHELL_CMD_ARG(status, NULL, "...", cmd_status, 1, 0);
SHELL_CMD_REGISTER(event, &sub_event, "...", NULL);
```

01 Kommandobaum

Bündelt statische Unterkommandos.

02 Argumentvertrag

Legt Handler und Argumentanzahl fest.

SHELL_CMD_REGISTER veröffentlicht das Root-Kommando und verbindet Hilfe mit dem Unterkommandobaum.

Argumente sicher übernehmen

Jede Eingabe wird vollständig geprüft, bevor sich der Anwendungszustand ändert

01

Anzahl

argc gegen Pflicht- und optionale Werte prüfen

02

Typ

Text vollständig in eine Zahl umwandeln

03

Bereich

Grenzen nach der Konvertierung prüfen

04

Überzählig

unerwartete Argumente klar ablehnen

05

Fehlerweg

Nutzung anzeigen und Zustand unverändert lassen

Ungültige Eingaben führen zu einer verständlichen Meldung und einem passenden negativen Rückgabewert.

Ausgaben und Rückgabewerte

HANDLER-AUSGABE

```
shell_print(sh, "status: ready");
shell_warn(sh, "already reset");
shell_error(sh, "invalid value");
return -EINVAL;
```

01 Normale Antwort

shell_print() meldet erfolgreiche Bedienergebnisse.

02 Warnung

shell_warn() kennzeichnet eine auffällige, aber verständliche Situation.

03 Fehler

shell_error() erklärt Ablehnung oder fehlgeschlagene Ausführung.

Shell-Ausgabe beantwortet die Bedienaktion. Asynchrone Systemereignisse bleiben im Logging.

Kurzer Handler, klare Delegation

HANDLER-SKIZZE

```
int rc = parse_and_validate(argc, argv);
if (rc < 0) { return rc; }

return event_service_request(action);
```

01 **Shell-Handler**

Eingaben prüfen, Fachfunktion aufrufen und Ergebnis berichten.

02 **Service oder Worker**

Längere oder asynchrone Arbeit außerhalb des Shell-Threads ausführen.

Gemeinsamer Zustand zwischen Shell und Ereignisverarbeitung benötigt eine definierte Synchronisation.

Entwurfscheck für den event-Kommandobaum

VERTRAG

Syntax vollständig

Unterkommandos benannt

Argumente und Werte definiert

Seiteneffekte beschrieben

FEHLERFÄLLE

Reaktion festgelegt

fehlend oder überzählig

ungültiger Typ oder Bereich

Gerät oder Service nicht bereit

ARCHITEKTUR

Verantwortung getrennt

Handler validiert und delegiert

Service besitzt Fachlogik

Worker übernimmt lange Arbeit

Das Design ist bereit für die Implementierung von status, stats und reset in 3.07.